Timed Assignment - Results

Attempt 1 of 1

Written Oct 24, 2026 3:30 PM - Oct 24, 2026 4:30 PM

Released Oct 27, 2026 9:30 PM

Attempt Score

Overall Grade (Highest Attempt)

For each blank below, fill in the expected output.

Python

```python
data = [7, 12, 5, 18, 11]

def mix(a, b):
    if a > b:
        return a - b
    else:
        return b - a + 1

def scan(items, x):
    for i in range(len(items)):
        if items[i] % x == 0:
            return i
    return -1

def crunch(items):
    result = 0
    for n in items:
        if n % 2 == 0:
            result = result + n // 2
        else:

```

```
22              result = result - 1
23      return result
24
25  print(mix(4, 9))
26  print(scan(data, 6))
27  print(crunch(data))
28  print(mix(crunch(data), scan(data, 2)))
29
30  final = []
31  for i in range(len(data)):
32      if data[i] % 9 == 0:
33          final.append((i, data[i]))
34
    print(final)
```

Question 1

4.004 / 7 points

What is the expected output for line 24? ___6___
What is the expected output for line 25? ___18___
What is the expected output for line 26? ___10___
What is the expected output for line 27? ___9___
What is the expected output for line 34? ___[(3,18)]___

The following code takes in `trainees`, as a list of tuples, and does some operations to it.

JavaScript

```
1   trainees = [
2       ("Antman", 71),
3       ("Hulk", 92),
4       ("Stark", 95),
5       ("Wanda", 83),
6       ("Barnes", 76),
7       ("Parker", 80),
8   ]
9
10  def get_successful_trainees(records):
11      return [name for name, score in records if score >= 80]
12
13
14  def build_rating_dict(records):
15      return {
16          name: "Great" if score >= 90 \
17
```

```
18          else "Good" if score >= 80 else "Redo"
19          for name, score in records
20      }
21
22  def sort_by_score(records):
23      return sorted(records, key=lambda x: x[1], reverse=True)
24
25
26  def get_top_scorers(records, n):
27      ranked = sort_by_score(records)
28      return [name for name, score in ranked[:n]]
29
30
31  passing = get_successful_trainees(trainees)
32  grades = build_rating_dict(trainees)
33  top_students = get_top_scorers(trainees, 2)
34
35  print(passing)
    print(top_students)
```

For each blank below, fill in the expected output from the specified line of code. For example, the expected output from `print([7, 8, 9])` is `[7, 8, 9]`.

The order of the returned items matters.

Question 2 — 2 / 4 points

What is the expected output from `print(passing)`?
___[("Hulk", 92), ("Stark", 95), ("Wanda", 85), ("Parker", 80)]___

What is the expected output from `print(top_students)`?
___[("Stark", 95), ("Hulk", 92)]___

Question 3 — 0 / 3 points

Using one single line (without any `;`), write a list comprehension to give a list of trainees whose ratings, stored in the variable `grades` and obtained from `build_rating_dict()`, correspond to 'Redo'.

Any answer beyond a single line would get 0 marks.

___lamda grade: grade=="Redo" for grade in grades___

Question 4

1 / 3 points

Hydra wants to mess things up, and creates a new function `build_correct_dict` (the word 'correct' means nothing).
Continuing from the previous cell, where `trainees` is already defined, will the code below execute successfully without a runtime error?

```
def build_correct_dict(records):
    holder = {}
    for name, score in records:
        if 90 <= score <= 100:
            holder[name] = "Great"
            return holder
        elif score < 90:
            holder[name] = "Good"
            return holder
        else:
            holder[name] = "Redo"
            return holder

print(build_correct_dict(trainees))
```

If an error occurs, identify the line that causes the error and briefly explain why.
If there is no runtime error, state what would be displayed from `print(build_correct_dict(trainees))`.

___The error line is "elif score < 90:". Hydra is using a binary option to sweep all trainees with less than 90 into the "good" category. It is a mtually exclusive outcome where the trainees with >= 90 marks will fall in "great" and everyone else will fall into "good". Hydra should have restricted it to "elif 80 <= score < 90:" for the elif to be valid.___

Hide question 4 feedback

Feedback

That is not a runtime error

In University-of-Numbers, everyone's transcript records only the marks, without any letter grade. Having 50 marks or more equates to a pass.

We have a list `students`, containing students' names (type `str`), and a dictionary `grades`, where the keys are course codes (type `str`), with the corresponding values being lists of integers for each student, stored in the same order as `students`.

For example, 'Di' scored 63, 48, 57, 84 and 61 marks, respectively.

Each student only likes courses in which he/she scores well in, regardless of what other students score relative to themselves. Complete the blanks, so that the code below gives the desired output. Specifically, given a `student_of_interest`, we want to know the corresponding `favourite_course` and `num_passes` for that student.

Python

```
students = ['Ang', 'Beh', 'Chan', 'Di', 'Eng']
grades = {
    'CS861': [76, 89, 45, 63, 52],
    'CS862': [56, 92, 59, 48, 63],
    'CS863': [83, 77, 60, 57, 29],
    'CS864': [90, 93, 72, 84, 65],
    'CS865': [49, 73, 36, 61, 78],
}

student_of_interest = 'Chan'
idx = _______<(a)>_______.index(student_of_interest)


highest_score = -1
num_passes = 0
favourite_course = ''

```

```
for course, scores in _______<(b)>_______:
    if scores[idx] >= 50:
        _______<(c)>_______
    if scores[idx] > highest_score:
        highest_score = _______<(d)>_______
        favourite_course = course

print(f"The favourite course of {student_of_interest} is _______<(e)>_______.

# Expected output to be displayed:
# The favourite course of Chan is CS864. This student passed 3 courses.
```

Question 5 — 4 / 5 points

Blank (a): ___students___
Blank (b): ___grades___
Blank (c): ___num_passes +=1___
Blank (d): ___scores[idx]___
Blank (e): ___{favourite_course}___

A fitness app records how many steps its user takes on each day of one week, together with a daily step goal. The program should print: the total steps for the week, the average per day (whole steps, rounded down), the best day's count, the first day the goal was reached, how many days met the goal, and finally one line for each day that hit the goal. Days are numbered from 0 (the list index).

Each function contains ONE error.
For each error:
state the line number, and write the corrected code for that line, you may ignore the indentation.
For lines 28–35, if a line is correct, fill in OK; if not, fill in the corrected code.

Note: There will be no syntax error.

Expected Output after running the code:

Total: 58000
Average: 8285
Best day: 12000
First goal day: 1
Goal days: 4
Day 1 hit the goal
Day 3 hit the goal
Day 5 hit the goal
Day 6 hit the goal

Python

```python
def total_steps(steps):

    for s in steps:

        total += s

    return total

def average_steps(steps):
    return sum(steps) / len(steps)

def first_goal_day(steps, goal):
    for i in range(len(steps)):
        if steps[i] >= goal:
            return steps[i]
    return -1

def count_goal_days(steps, goal):
    count = 0
    for s in steps:
        if s > goal:
            count += 1
    return count

steps = [4500, 9200, 7800, 12000, 6100, 10400, 8000]
goal = 8000

print("Total:" + total_steps(steps))
print("Average:", average_steps(steps))
print("Best day:", min(steps))
print("First goal day:", first_goal_day(steps, goal))
print("Goal days:", count_goal_days(steps))
for i in range(len(steps) - 1):
    if steps[i] >= goal:
        print("Day", i, "hit the goal")
```

Question 6 1 / 1 point

Which line in the function `total_steps` needs correction? ___2___
The corrected line is: ___total = 0___

Question 7 0.3 / 1 point

Which line in the function `average_steps` needs correction? ___10___
The corrected line is: ___return (sum(steps) / len(steps))___

Question 8 0 / 1 point

Which line in the function `first_goal_day` needs correction? ___10___
The corrected line is: ___break___

Question 9 1 / 1 point

Which line in the function `count_goal_days` needs correction? ___21___
The corrected line is: ___if s >= goal:___

Question 10 3.8 / 4 points

For lines 28–35, if a line is correct, fill in 'OK'; if not, fill in the corrected code.

line 28 ___print("Total: ", total_steps(steps))___
line 29 ___OK___
line 30 ___print("Best day:", max(steps))___
line 31 ___OK___
line 32 ___print("Goal days:", count_goal_days(steps, goal))___
line 33 ___for i in range(len(steps)):___
line 34 ___OK___
line 35 ___print("Day {} hit the goal".format(i))___

Done